VAPT Report

Vulnerability Assessment & Penetration Testing

DVWA (Damn Vulnerable Web Application) Security Assessment

Client	Metasploitable2 Virtual Machine (Testing Environment)
Application	DVWA (Damn Vulnerable Web Application)
Assessment Date	May 15-16, 2026

Table of Contents

1. Web Application Penetration Testing

1.1 Executive Summary

The Security Testing Team carried out a detailed white-box penetration test on the Damn Vulnerable Web Application (DVWA) hosted within the Metasploitable2 environment during May 15–16, 2026. This engagement was designed to uncover exploitable weaknesses and misconfigured security controls. The evaluation was conducted in alignment with recognized industry standards and frameworks, including the OWASP Top 10, OWASP ASVS, SANS, and NIST guidelines.

Overall Risk Level: HIGH

The assessment revealed **21 distinct vulnerabilities** across varying severity levels. Of immediate concern are **2 High-severity** and **6 Medium-severity** issues that expose the organization to significant risk. Successful exploitation of these weaknesses could enable adversaries to:

- Gain unauthorized, full access to the application's database.
- Execute arbitrary commands on the underlying operating system.
- Capture user session credentials and impersonate legitimate accounts.
- Alter, delete, or exfiltrate sensitive business information.
- Disrupt the integrity and availability of the overall system

Key Findings:
The following table summarizes the vulnerabilities discovered during this assessment:

Severity	Count	Percentage	CVSS Range	Priority
High	2	30%	9.0 - 10.0	P1 - Immediate
Medium	6	20%	7.0 - 8.9	P2 - Urgent
Low	8	30%	4.0 - 6.9	P3 - Important
Informational	5	20%	0.1 - 3.9	P4 - Standard
Total	**21**	**100%**	-	-

Critical Recommendations:

- **Immediate (0–24 hours):** Prioritize resolution of all **High-severity vulnerabilities**, with urgent focus on **SQL Injection** and **Command Injection** flaws. Apply emergency patches and activate enhanced monitoring to detect potential exploitation attempts.
- **Short-Term (1–7 days):** Address **Medium-severity findings**, including **Cross-Site Scripting (XSS)** and insufficient **CSRF protections**. Implement robust input validation and output encoding across all user-supplied data to mitigate these risks.
- **Medium-Term (1–4 weeks):** Remediate **ow-severity issues** by strengthening overall security posture. Recommended measures include deploying a **Web Application Firewall (WAF)**, enforcing **security headers**, and improving **session management controls**.
- **Long-Term (1–3 months):** Establish a **Secure Development Lifecycle (SDLC)** to integrate security into every stage of application development. Provide **security awareness training** for the development team and implement **automated security testing** within the CI/CD pipeline to ensure ongoing protection.

1.2 Scope of Assessment

The scope for this Security assessment included comprehensive testing across multiple attack vectors:

• Information Gathering and Reconnaissance

• Configuration Management Testing

• Business Logic Testing

• Authentication Mechanism Analysis

• Authorization and Access Control Testing

• Session Management Security

• Data Validation, Governance, and Transfer Security

Application Name	DVWA (Damn Vulnerable Web Application)
URL	http://192.168.196.129/dvwa/
Environment	Metasploitable 2 VM (Isolated Lab setup)
IP Address	192.168.196.129
Authentication Method	Form-based Login
Backend Database	MySQL
Test Account	admin / password

1.3 Out of Scope

The following activities were explicitly excluded from this engagement:

- Social Engineering attacks
- Denial of Service (DoS) testing
- Physical security assessment
- Source code review (white-box analysis)
- Network infrastructure penetration testing
- Operating system level vulnerabilities

1.4 Tools Used

The assessment utilized industry-standard tools combined with manual testing techniques:

- **Burp Suite Professional 2025.x** - Web application security testing and traffic interception
- **SQLMap v1.7.x** - Automated SQL injection detection and exploitation
- **OWASP ZAP v2.14.x** - Automated vulnerability scanning and fuzzing
- **Nmap v7.94** - Network reconnaissance and service enumeration
- **Kali Linux 2025.x** - Primary penetration testing platform
- **Custom Python Scripts** - Targeted testing and proof-of-concept development

2. Vulnerability Details

2.1 SQL Injection in User ID Parameter

Severity:	HIGH
CVSS v3.1 Score:	9.9 (Very Critical)
CVSS Vector:	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CWE:	CWE-89: Improper Neutralisation of Special Elements in SQL
Affected URL:	http://192.168.196.129/dvwa/vulnerabilities/sqli/
Parameter:	id (GET)
Discovery Date:	May 16, 2026

Description:

The application's SQL Injection module does not adequately sanitize user-supplied input in the id **parameter** before incorporating it into SQL queries. This flaw enables attackers to manipulate query logic and execute arbitrary SQL commands.
The root cause is the reliance on **string concatenation** when constructing SQL queries, without applying **parameterized statements** or proper **input validation**. As a result, the database is exposed to unauthorized access, data manipulation, and potential compromise of system integrity.

Steps to Reproduce –– SQL Injection in id Parameter:

1. Navigate to: http://192.168.196.129/dvwa/vulnerabilities/sqli/
2. Set DVWA security level to **Low**.
3. In the **User ID** field, enter: 1'
    - Observe the SQL syntax error message, confirming the injection point.
4. Enter payload: 1' OR '1'='1
    - All users are returned instead of a single user.

5. Extract database name using:

```
1' UNION SELECT NULL,database()-- -
```

6. Enumerate tables with:

```
1' UNION SELECT NULL,table_name FROM information_schema.tables WHERE table_schema='dvwa'-- -
```

7. Extract user credentials using:

```
1' UNION SELECT user,password FROM users-- -
```

8. Successfully retrieved usernames and **MD5 password hashes**.

Technical Details:

```
Vulnerable Code Pattern (Estimated):

<?php
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = mysql_query($query);
?>
```

Impact Assessment:

Confidentiality Impact: HIGH Complete database compromise could expose all user credentials, personal information, and application data.

Integrity Impact: HIGH Attackers may modify, insert, or delete database records, leading to corruption of critical business data

.

Availability Impact: HIGH Exploitation could result in dropped tables, corrupted databases, or resource-intensive queries that render the application non-functional.

Business Impact

- Regulatory compliance violations (**GDPR, PCI-DSS, HIPAA**)
- Reputational damage due to public disclosure of a data breach
- Financial losses from fraud, remediation costs, and regulatory fines
- Legal liability arising from compromised customer data
- Loss of customer trust and potential disruption of business continuity

Remediation- SQL Injection Vulnerability:

Immediate Actions (Priority P0 – Within 24 hours):

- Deploy a Web Application Firewall (WAF) with SQL injection signatures.
- Implement temporary input validation to reject special SQL characters.
- Enable comprehensive database query logging and monitoring.
- Review and restrict database user permissions to enforce least privilege.
- Isolate the vulnerable module from production, if feasible.

Permanent Fix (Within 1 week):

Implement parameterized queries (prepared statements) to eliminate SQL injection vulnerabilities.

```
 <?php
$id = $_GET['id'];
// Sanitize input
$id = intval($id); // Ensure integer type

// Use prepared statement
$stmt = $db->prepare("SELECT first_name, last_name FROM users WHERE user_id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$result = $stmt->get_result();
?>
```

Additional Security Controls:

- Apply **whitelist-based input validation** for all user inputs.
- Use **Object-Relational Mapping (ORM)** frameworks (e.g., PDO, Eloquent).
- Enable **Runtime Application Self-Protection (RASP)** for SQL injection detection.
- Implement **database activity monitoring and alerting**.
- Conduct **security code reviews** of all database interaction code.
- Deploy **Static Application Security Testing (SAST)** and **Dynamic Application Security Testing (DAST)** tools in the CI/CD pipeline.
- Enforce the **principle of least privilege** for database accounts.
- Remove detailed error messages from the production environment.

3. Testing Methodology

The penetration testing engagement was conducted using a **white-box methodology**, leveraging prior knowledge of the application and its environment to ensure comprehensive coverage. The structured process included:

- **Information Gathering & Reconnaissance:** Identifying application entry points, mapping the attack surface, and collecting system details.
- **Configuration Management Testing:** Reviewing application and database configurations for insecure defaults and misconfigurations.
- **Business Logic Testing:** Assessing workflows and logic for flaws that could be exploited to bypass intended functionality.
- **Authentication & Authorization Analysis:** Evaluating login mechanisms, credential handling, and access control enforcement.
- **Session Management Security:** Testing session handling, cookie security, and timeout mechanisms.
- **Data Validation & Transfer Security:** Examining input validation, sanitization, and secure data transmission practices.
- **Exploitation & Post-Exploitation:** Safely validating vulnerabilities to demonstrate real-world impact, including SQL Injection and Command Injection flaws.
- **Reporting:** Documenting findings with severity ratings, CVSS scoring, and prioritized remediation recommendations.

This methodology aligns with **OWASP Top 10, OWASP ASVS, SANS, and NIST guidelines**, ensuring adherence to industry best practices.

4. Risk Assessment

The vulnerabilities identified were classified using the **Confidentiality, Integrity, and Availability (CIA)** triad and mapped to severity levels based on CVSS v3.1 scoring:

- **High Severity (2 findings, CVSS 9.0–10.0):** Immediate exploitation risk, including SQL Injection and Command Injection flaws.
- **Medium Severity (6 findings, CVSS 7.0–8.9):** Significant risks such as Cross-Site Scripting (XSS) and insufficient CSRF protections.
- **Low Severity (8 findings, CVSS 4.0–6.9):** Issues with moderate impact, including weak session management and missing security headers.
- **Informational (5 findings, CVSS 0.1–3.9):** Observations with minimal direct impact but recommended for best practices.

Overall Risk Level: HIGH Exploitation of these vulnerabilities could allow adversaries to:

- Gain unauthorized access to the database.
- Execute arbitrary system commands.
- Steal user credentials and impersonate accounts.
- Modify, delete, or exfiltrate sensitive data.
- Disrupt application availability and business continuity.

This assessment provides a **prioritized roadmap for remediation**, ensuring that immediate threats are addressed first while planning for long-term improvements.

5. Terms and Conditions

This report is **confidential** and intended solely for the client organization. The findings and recommendations are based on the scope and environment provided during testing (DVWA hosted on Metasploitable2).

- The Security Testing Team is **not liable** for changes made to the environment after the assessment.
- The client is responsible for **implementing remediation measures** and maintaining ongoing security.
- Unauthorized distribution, disclosure, or use of this report is **strictly prohibited** and may violate applicable laws.
- The assessment was conducted under **controlled lab conditions**; exploitation in a live production environment may yield different outcomes.
- The scope excluded **social engineering, denial-of-service testing, physical security, source code review, network infrastructure testing, and OS-level vulnerabilities**.

****